```
        hmemdc = CreateCompatibleDC ( hdc ) ;
        holdbmp = SelectObject ( hmemdc, hbmp ) ;

        ReleaseDC ( hWnd, hdc ) ;

        srand ( time ( NULL ) ) ;

        GetClientRect ( hWnd, &r ) ;

        x = rand( ) % r.right - 22 ;
        y = rand( ) % r.bottom - 22 ;

        SetTimer ( hWnd, 1, 50, NULL ) ;
}

void OnDestroy ( HWND hWnd )
{
        KillTimer ( hWnd, 1 ) ;
        SelectObject ( hmemdc, holdbmp ) ;
        DeleteDC ( hmemdc ) ;
        DeleteObject ( hbmp ) ;
        PostQuitMessage ( 0 ) ;
}

void OnTimer ( HWND hWnd )
{
        HDC hdc ;
        RECT r ;
        const int wd = 22, ht = 22 ;
        static int dx = 10, dy = 10 ;

        hdc = GetDC ( hWnd ) ;
        BitBlt ( hdc, x, y, wd, ht, hmemdc, 0, 0, WHITENESS ) ;
        GetClientRect ( hWnd, &r ) ;

        x += dx ;
        if ( x < 0 )
        {
```

```
            x = 0 ;
            dx = 10 ;
            PlaySound ("chord.wav", NULL, SND_FILENAME | SND_ASYNC ) ;
        }
        else if ( x > ( r.right - wd ) )
        {
            x = r.right - wd ;
            dx = -10 ;
            PlaySound ("chord.wav", NULL, SND_FILENAME | SND_ASYNC ) ;
        }

        y += dy ;
        if ( y < 0 )
        {
            y = 0 ;
            dy = 10 ;
            PlaySound ("chord.wav", NULL, SND_FILENAME | SND_ASYNC ) ;
        }
        else if ( y > ( r.bottom - ht ) )
        {
            y = r.bottom - ht ;
            dy = -10 ;
            PlaySound ( "chord.wav", NULL, SND_FILENAME | SND_ASYNC );
        }

        BitBlt ( hdc, x, y, wd, ht, hmemdc, 0, 0, SRCCOPY ) ;
        ReleaseDC ( hWnd, hdc ) ;
}
```

From the **WndProc( )** function you can observe that we have handled two new messages here—**WM_CREATE** and **WM_TIMER**. For these messages we have called the handlers **OnCreate( )** and **OnTimer( )** respectively. Let us now understand these handlers one by one

WM_CREATE and *OnCreate( )*

The **WM_CREATE** message arrives whenever a new window is created. Since usually a window is created only once, the one-time activity that is to be carried out in a program is usually done in **OnCreate( )** handler. In our program to make the ball move we need to display it at different places at different times. To do this it would be necessary to blit the ball image several times. However, we need to load the image only once. As this is a one-time activity it has been done in the handler function **OnCreate( )**.

You are already familiar with the steps involved in preparing the image for blitting—loading the bitmap, creating a memory DC, making it compatible with screen DC and selecting the bitmap in the memory DC.

Apart from preparing the image for blitting we have also done some intialisations like setting up values in some variables to indicate the initial position of the ball. We have also called the **SetTimer( )** function. This function tells Windows to post a message **WM_TIMER** into the message queue of our application every 50 milliseconds.

WM_TIMER and *OnTimer( )*

If we are to perform an activity at regular intervals we have two choices:

(a) Use a loop and monitor within the loop when is it time to perform that activity.
(b) Use a Windows mechanism of timer. This mechanism when used posts a WM_TIMER message at regular intervals to our application.

The first method would seriously hamper the responsiveness of the program. If the control is within the loop and a new message arrives the message would not get processed unless the control goes out of the loop. The second choice is better because it makes the program event driven. That is, whenever **WM_TIMER** arrives

that time its handler does the job that we want to get executed periodically. At other times the application is free to handle other messages that come to its queue.

All that we have done in the **OnTimer( )** handler is erase the ball from previous position and draw it at a new position. We have also checked if the ball has hit the boundaries of the window. If so we have played a sound file using the **PlaySound( )** API function and then changed the direction of the ball.

A Few More Points…

A few more points worth noting before we close our discussion on animation…

(a) One application can set up multiple timers to do different jobs at different intervals. Hence we need to pass the id of the timer that we want to set up to the **SetTimer( )** function. In our case we have specified the id as 1.

(b) For multiple timers Windows would post multiple **WM_TIMER** messages. Each time it would pass the timer id as additional information about the message.

(c) For drawing as well as erasing the ball we have used the same function—**BitBlt( )**. While erasing we have used the raster operation code **WHITENESS**. When we use this code the color values of the source pixels get ignored. Thus red colored pixels of ball would get ignored leading to erasure of the ball in the window.

(d) The size of client area of the window can be obtained using the **GetClientRect( )** API function.

(e) We want that every time we run the application the initial position of the ball should be different. To ensure this we have generated its initial x, y coordinates using the standard library function **rand( )**. However, this function doesn't

generate true random numbers. To ensure that we do get true random numbers, somehow we need to tie the random number generation with time, as time of each execution of our program would be different. This has been achieved by making the call

```
srand ( time ( NULL ) ) ;
```

Here **time( )** is function that returns the time. We have further passed this time to the **srand( )** function.

(f) To be able to use **rand( )** and **srand( )** functions include the file 'stdlib.h'. Similarly for **time( )** function to work include the file 'time.h'.

(g) In the call to the **PlaySound( )** function the first parameter is the name of the wave file that is to be played. If first parameter is filename then the second has to be NULL. The third parameter is a set of flags. **SND_FILENAME** indicates that the first parameter is the filename, whereas **SND_ASYNC** indicates that the sound should be played in the background.

(h) To be able to use the **PlaySound( )** function we need to link the library 'winmm.lib'. This is done by using 'Project | Settings' menu item. On selection of this item a dialog pops up. In the 'Link' tab of this dialog mention the name 'winmm.lib' in the 'Object / Library modules' edit box.

(i) When the application terminates we have to instruct Windows not to send **WM_TIMER** messages to our application any more. For this we have called the **KillTimer( )** API function passing to it the ID of the timer.

Windows, the Endless World…

The biggest hurdle in Windows programming is a sound understanding of its programming model. In this chapter and in the

last two I have tried to catch the essence of Windows' Event Driven Programming model. Once you have understood it thoroughly rest is just a matter of understanding and calling the suitable API functions to get your job done. Windows API is truly an endless world. It covers areas like Networking, Internet programming, Telephony, Drawing and Printing, Device I/O, Imaging, Messaging, Multimedia, Windowing, Database programming, Shell programming, to name a few. The programs that we have written have merely scratched the surface. No matter how many programs that we write under Windows, several still remain to be written. The intention of this chapter was to unveil before you, to give you the first glimpse of what is possible under Windows. The intention all along was not to catch fish for you but to show you how to catch fish so that you can do fishing all your life. Having made a sound beginning, rest is for you to explore. Good luck and happy fishing!

Summary

(a) In DOS, programmers had to write separate graphics code for every new video adapter. In Windows, the code once written works on any video adapter.

(b) A Windows program cannot draw directly on an output device like screen or printer. Instead, it draws to the logical display surface using device context.

(c) When the window is displayed for the first time, or when it is moved or resized **OnPaint( )** handler gets called.

(d) It is necessary to obtain the device context before drawing text or graphics in the client area.

(j) A device context is a structure containing information required to draw on a display surface. The information includes color of pen and brush, screen resolution, color palettes, etc.

(e) To draw using a new pen or brush it is necessary to select them into the device context.

(f) If we don't select any brush or pen into the device context then the drawing drawn in the client area would be drawn with the default pen (black pen) and default brush (white brush).
(g) RGB is a macro representing the Red, Green and Blue elements of a color. RGB (0, 0, 0) gives black color, whereas, RGB (255, 255, 255) gives white color.
(h) Animation involves repeatedly drawing the same image at successive positions.

Exercise

[A] State True or False:

(a) Device independence means the same program is able to work using different screens, keyboards and printers without modifications to the program.
(b) The WM_PAINT message is generated whenever the client area of the window needs to be redrawn.
(c) The API function EndPaint( ) is used to release the DC.
(d) The default pen in the DC is a solid pen of white color.
(e) The pen thickness for the pen style other than PS_SOLID has to be 1 pixel.
(f) BeginPaint( ) and GetDC( ) can be used interchangeably.
(g) If we drag the mouse from (10, 10) to (110, 100), 100 WM_MOUSEMOVE messages would be posted into the message queue.
(h) WM_PAINT message is raised when the window contents are scrolled.
(i) With each DC a default monochrome bitmap of size 1 pixel x 1 pixel is associated.
(j) The WM_CREATE message arrives whenever a window is displayed.

[B] Answer the following:

(a) What is meant by Device Independent Drawing and how it is achieved?

(b) Explain the significance of WM_PAINT message.

(c) How Windows manages the code and various resources of a program?

(d) Explain the Windows mechanism of timer.

(e) What do you mean by capturing a mouse?

(f) Write down the steps that need to be carried out to animate an object.

[C] Attempt the following:

(a) Write a program, which displays "hello" at any place in the window where you click the left mouse button. If you click the right mouse button the color of subsequent hellos should change.

(b) Write a program that would draw a line by joining the new point where you have clicked the left mouse button with the last point where you clicked the left mouse button.

(c) Write a program to gradient fill the entire client area with shades of blue color.

(d) Write a program to create chessboard like boxes (8 X 8) in the client area. If the window is resized the boxes should also get resized so that all the 64 boxes are visible at all times.

(e) Write a program that displays only the upper half of a bitmap of size 40 x 40.

(f) Write a program that displays different text in different colors and fonts at different places after every 10 seconds.

19 Interaction With Hardware

- Hardware Interaction
- Hardware Interaction, DOS Perspective
- Hardware Interaction, Windows Perspective
- Communication with Storage Devices
  - The *ReadSector( )* Function
- Accessing Other Storage Devices
- Communication with Keyboard
  - Dynamic Linking
  - Windows Hooks
- Caps Locked, Permanently
- Did You Press It TTwwiiccee….
- Mangling Keys
- KeyLogger
- Where is This Leading
- Summary
- Exercise

There are two types of Windows programmers those who are happy in knowing the things the way they are under Windows and those who wish to know why the things are the way they are. This chapter is for the second breed of programmers. They are the real power users of Windows. Because it is they who first understand the default working of different mechanisms that Windows uses and then are able to make those mechanisms work to their advantage. The focus here would be restricted to mechanisms that are involved in interaction with the hardware under the Windows world. Read on and I am sure you would be on your path to become a powerful Windows programmer.

Hardware Interaction

Primarily interaction with hardware suggests interaction with peripheral devices. However, its reach is not limited to interaction with peripherals. The interaction may also involve communicating with chips present on the motherboard. Thus more correctly, interaction with hardware would mean interaction with any chip other than the microprocessor. During this interaction one or more of the following activities may be performed:

(a) Reacting to events that occur because of user's interaction with the hardware. For example, if the user presses a key or clicks the mouse button then our program may do something.

(b) Reacting to events that do not need explicit user's interaction. For example, on ticking of a timer our program may want to do something.

(c) Explicit communication from a program without the occurrence of an event. For example, a program may want to send a character to the printer, or a program may want to read/write the contents of a sector from the hard disk.

Let us now see how this interaction is done under different platforms.

Hardware Interaction, DOS Perspective

Under DOS whenever an external event (like pressing a key or ticking of timer) occurs a signal called hardware interrupt gets generated. For different events there are different interrupts. As a reaction to the occurrence of an interrupt a table called Interrupt Vector Table (IVT) is looked up. IVT is present in memory. It is populated with addresses of different BIOS routines during booting. Depending upon which interrupt has occurred the Microprocessor picks the address of the appropriate BIOS routine from IVT and transfers execution control to it. Once the control reaches the BIOS routine, the code in the BIOS routine interacts with the hardware. Naturally, for different interrupts different BIOS routines are called. Since these routines serve the interrupts they are often called 'Interrupt Service Routines' or simply ISRs.

Refer Figure 19.1 to understand this mechanism.

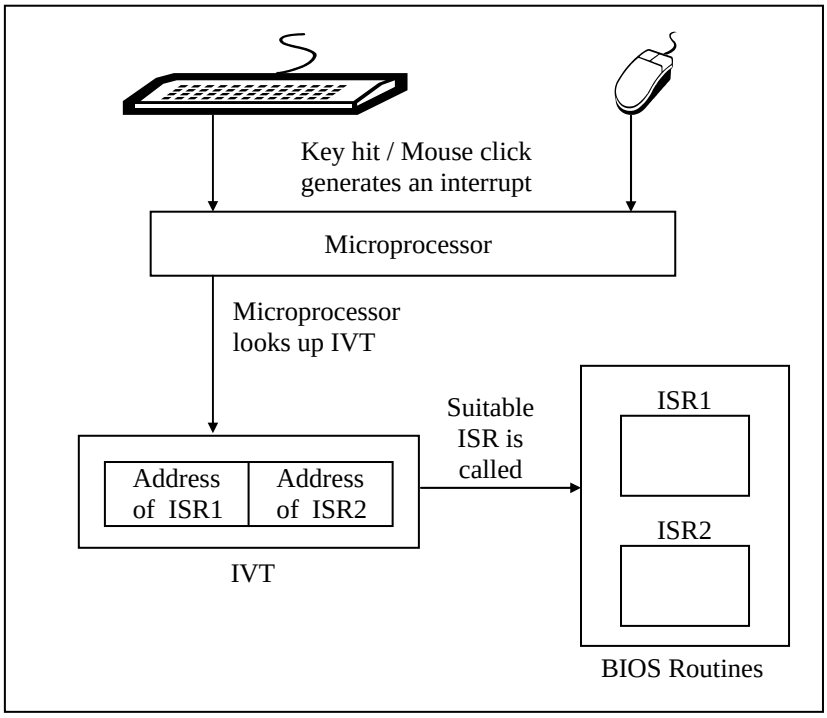

Figure 19.1

If we want that instead of the default ISR our routine should get called then it is necessary to store the address of this routine in IVT. Once this is done whenever a hardware interrupt occurs our routine's address from IVT is picked up and the control is transferred to our routine. For example, we may register our ISR in IVT to gain finer control over the way key-hits from the keyboard are tackled. This finer control may involve changing codes of keys or handling hitting of multiple keys simultaneously.

Explicit communication with the hardware can be done in four different ways. These are shown in Figure 19.2.

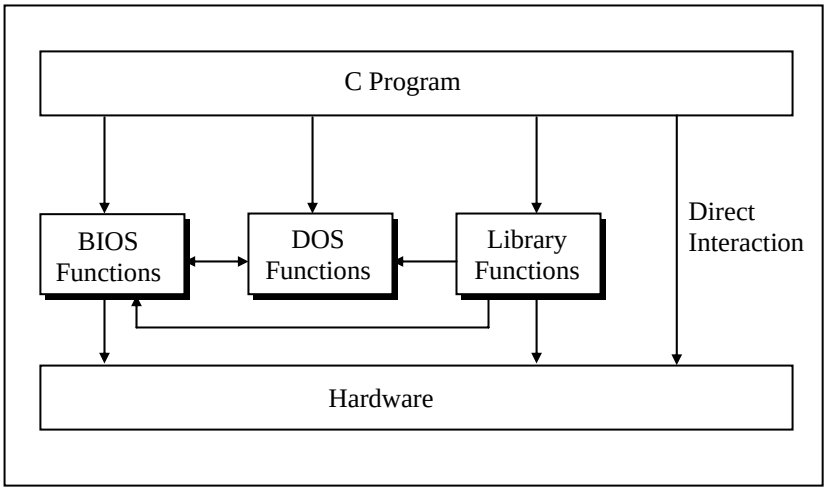

Figure 19.2

Let us now discuss the pros and cons of using these different methods to interact with the hardware.

(a) Calling DOS Functions

To interact with the hardware a program can call DOS functions. These functions can either directly interact with the hardware or they may call BIOS functions which in turn interact with the hardware. As a result, the programmer is not required to know all the hardware details to be able to interact with it. However, since DOS functions do not have names they have to be called through the mechanism of interrupts. This is difficult since the programmer has to remember interrupt service numbers for calling different DOS functions. Moreover, communication with these functions has to be done using CPU registers. This leads to lot of difficulties since different functions use different registers for communication. So one has to know details of different CPU registers, how to use them, which one to use when, etc.

(b) Calling BIOS Functions

DOS functions can carry out jobs like console I/O, file I/O, printing, etc. For other operations like generating graphics, carrying out serial communication, etc. the program has to call another set of functions called ROM-BIOS functions. Note that there are some functions in ROM-BIOS that do same jobs as equivalent DOS functions. BIOS functions suffer from the same difficulty as DOS functions—they do not have names. Hence they have to be called using interrupts and involve heavy usage of registers.

(c) Calling Library Functions

We can call library functions which in turn can call DOS/BIOS functions to carry out the interaction with hardware. Good examples of these functions are **printf( ) / scanf( ) / getch( )** for interaction with console, **absread( ) / abswrite( )** for interaction with disk, **bioscom( )** for interaction with serial port, etc. But the library doesn't have a parallel function for every DOS/BIOS function. Hence at some point of time one has to learn how to call DOS/BIOS functions.

(d) Directly interacting with the hardware

At times the programs are needed to directly interact with the hardware. This has to be done because either there are no library functions or DOS/BIOS functions to do this, or if they are there their reach is limited. For example, while writing good video games one is required to watch the status of multiple keys simultaneously. The library functions as well as the DOS/BIOS functions are unable to do this. At such times we have to interact with the keyboard controller chip directly.

However, direct interaction with the hardware is difficult because one has to have good knowledge of technical details of the chip to be able to do so. Moreover, not every technical detail about how the hardware from a particular manufacturer works is well documented.

Hardware Interaction, Windows Perspective

Like DOS, under Windows too a hardware interrupt gets generated whenever an external event occurs. As a reaction to this signal a table called Interrupt Descriptor Table (IDT) is looked up and a corresponding routine for the interrupt gets called. Unlike DOS the IDT contains addresses of various kernel routines (instead of BIOS routines). These routines are part of the Windows OS itself. When the kernel routine is called, it in turn calls the ISR present in the appropriate device driver. This ISR interacts with the hardware. Two questions may now occur to you:

(a) Why the kernel routine does not interact with the hardware directly?
(b) Why the ISR of the device driver not registered directly in the IDT?

Let us find answer to the first question. Every piece of hardware works differently than the other. As new pieces of hardware come into existence new code has to be written to be able to interact with them. If this code is written in the kernel then the kernel would have to be rewritten and recompiled every time a new hardware comes into existence. This is practically impossible. Hence the new code to interact with the device is written in a separate program called device driver. With every new piece of hardware a new device driver is provided. This device driver is an extension of the OS itself.

Let us now answer the second question. Out of the several components of Windows OS a component called kernel is tightly integrated with the processor architecture. If the processor architecture changes then the kernel is bound to change. One of goals of Windows NT family was to keep the other components of OS and the device drivers portable across different microprocessor architectures. All processor architectures may not use IDT for the registration and lookup mechanism. So, had registration of the device driver's ISR in IDT been allowed, then the mechanism

would fail on processors which do not use IDT, thereby compromising portability of device drivers.

Refer Figure 19.3 for understanding the interrupt handling mechanism under Windows.

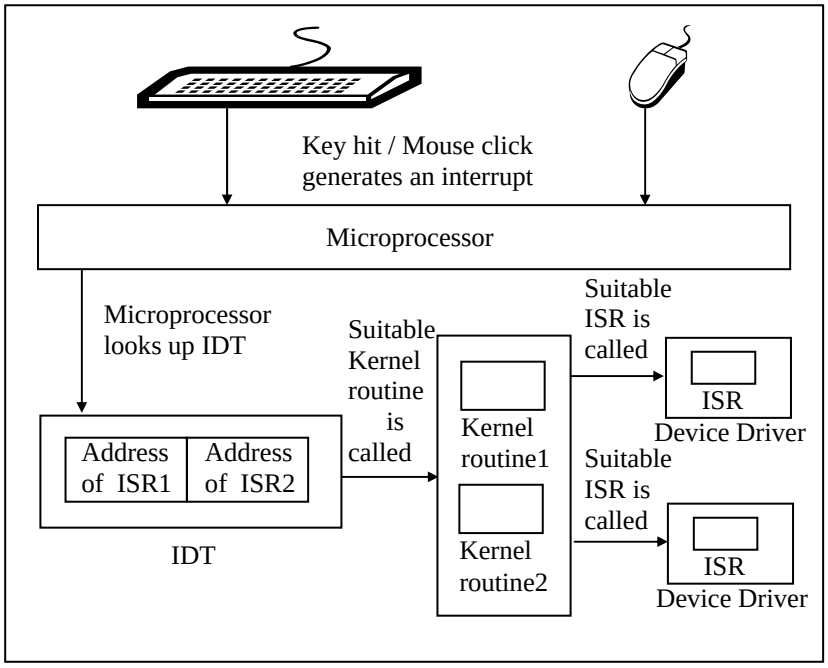

Figure 19.3

If we are to gain finer control while reacting to interrupts we would be required to write a device driver containing a new ISR to do so.

Under Windows explicit communication with hardware is much different than the way it was done under DOS. This is primarily because under Windows every device is shared amongst multiple applications running in memory. To avoid conflict between different programs accessing the same device simultaneously

Windows does not permit an application program to directly access any of the devices. Instead it provides several API functions to carry out the interaction. These functions have names so calling them is much easier than calling DOS/BIOS functions. When we call an API function to interact with a device, it in turn accesses the device driver program for the device. It is the device driver program that finally accesses the device. There is a standard way in which an application can communicate with the device driver. It is device driver's responsibility to ensure that multiple requests coming from different applications are handled without causing any conflict. In the sections to follow we would see how to communicate with the device driver to be able to interact with the hardware.

One last question—won't the API change if a new device comes into existence? No it won't. That is the beauty of the Windows architecture. All that would change is the device driver program for the new device. The API functions that we would need to interact with this new device driver would remain same. This is shown in Figure 19.4

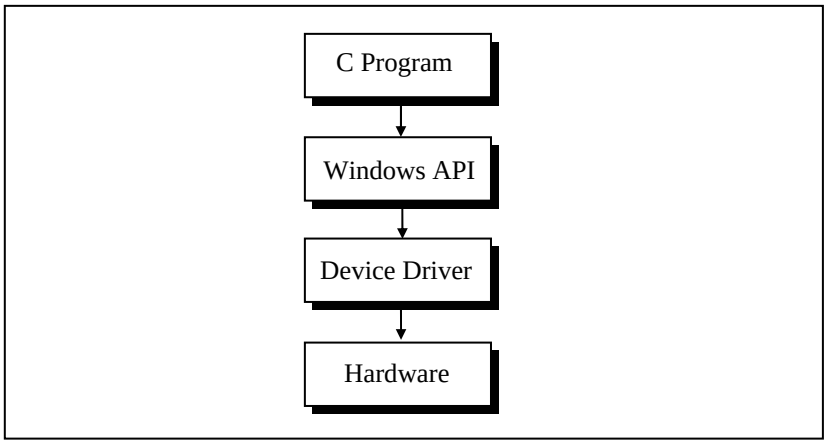

Figure 19.4

Communication with Storage Devices

Since DOS is commercially dead the rest of the chapter would focus on communication with the devices under Windows platform. We would illustrate this with the help of several programs.

Let us begin with the one that interacts with the simplest storage device, namely the floppy disk. Rather than the physical structure of the floppy disk it is the way the stored information is laid out and managed that concerns programmers most. Let us understand how the information is laid out on a floppy disk. Each floppy disk consists of four logical parts—Boot Sector, File Allocation Table (FAT), Directory and Data space. Of these, the Boot Sector contains information about how the disk is organized. That is, how many sides does it contain, how many tracks are there on each side, how many sectors are there per track, how many bytes are there per sector, etc. The files and the directories are stored in the Data Space. The Directory contains information about the files like its attributes, name, size, etc. The FAT contains information about where the files and directories are stored in the data space. Figure 19.5 shows the four logical parts of a 1.44 MB disk.

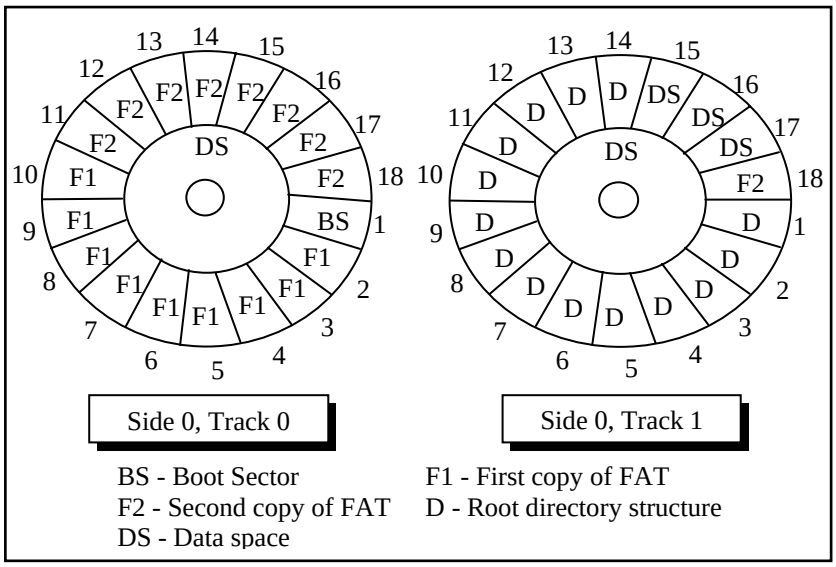

Figure 19.5

With the logical structure of the floppy disk behind us let us now write a program that reads the boot sector of a floppy disk and displays its contents on the screen. But why on earth would we ever like to do this? Well, that's what all Windows-based Anti-viral softwares do when they scan for boot sector viruses. A good enough reason for us to add the capability to read a boot sector to our knowledge! Here is the program…

```
# include <stdafx.h>
# include <windows.h>
# include <stdio.h>
# include <conio.h>

# pragma pack ( 1 )
struct boot
{
     BYTE jump [ 3 ] ;
```

```
    char bsOemName [ 8 ] ;
    WORD bytesperSector ;
    BYTE sectorspercluster ;
    WORD sectorsreservedarea ;
    BYTE copiesFAT ;
    WORD maxrootdirentries ;
    WORD totalSectors ;
    BYTE mediaDescriptor ;
    WORD sectorsperFAT ;
    WORD sectorsperTrack ;
    WORD sides ;
    WORD hiddenSectors ;
    char reserve [ 480 ] ;
} ;

void ReadSector ( char*src, int ss, int num, void* buff  ) ;

void main( )
{
    struct boot b ;
    ReadSector ( "\\\\.\\A:", 0, 1, &b ) ;

    printf ( "Boot Sector name: %s\n", b.id ) ;
    printf ( "Bytes per Sector: %d\n", b.bps )  ;
    printf ( "Sectors per Cluster: %d\n", b.spc ) ;
    /* rest of the statements can be written by referring Figure 19.6
       and Appendix G*/
}

void ReadSector ( char *src, int ss, int num, void* buff )
{
    HANDLE h ;
    unsigned int br ;
    h = CreateFile  ( src, GENERIC_READ,
                      FILE_SHARE_READ, 0, OPEN_EXISTING, 0, 0 ) ;
    SetFilePointer ( h, ( ss * 512 ), NULL, FILE_BEGIN ) ;
    ReadFile ( h, buff, 512 * num, &br, NULL )  )
    CloseHandle ( h ) ;
```

}

The boot sector contains two parts—'Boot Parameters' and 'Disk Bootstrap Program'. The Boot Parameters are useful while performing read/write operations on the disk. Figure 19.6 shows the break up of the boot parameters for a floppy disk.

Description	**Length**	**Typical Values**
Jump instruction	3	EB3C90
OEM name	8	MSWIN4.1
Bytes per sector	2	512
Sectors per cluster	1	1
Reserved sectors	2	1
Number of FAT copies	1	2
Max. Root directory entries	2	224
Total sectors	2	2880
Media descriptor	1	F0
Sectors per FAT	2	9
Sectors per track	2	18
No. Of sides	2	2
Hidden sectors	4	0
Huge sectors	4	0
BIOS drive number	1	0
Reserved sectors	1	0
Boot signature	1	41
Volume ID	4	349778522
Volume label	11	ICIT
File system type	8	FAT12

Figure 19.6

Using the breakup of bytes shown in Figure 19.6 our program has first created a structure called **boot**. Notice the usage of **#pragma pack** to ensure that all elements of the structure are aligned on a 1-byte boundary, rather than the default 4-byte boundary. Then comes the surprise—there is no **WinMain( )** in the program. This is because we want to display the boot sector contents on the screen rather than in a window. This has been done only for the sake of simplicity. Remember that our aim is to interact with the floppy, and not in drawing and painting in a window. If you wish you can of course adapt this program to display the same contents in a window. So the program is still a Windows application. Only difference is that it is built as a 'Win32 Console Application' using VC++. A console application always begins with **main( )** rather than **WinMain( )**.

To actually read the contents of boot sector of the floppy disk the program makes a call to a user-defined function called **ReadSector( )**. The **ReadSector( )** function is quite similar to the **absread( )** library function available in Turbo C/C++ under DOS.

The first parameter passed to **ReadSector( )** is a string that indicates the storage device from where the reading has to take place. The syntax for this string is **\\machine-name\storage-device name**. In **\\.\A:**, we have used '.' for the machine name. A '.' means the same machine on which the program is executing. Needless to say, **A:** refers to the floppy drive. The second parameter is the logical sector number. We have specified this as 0 which means the boot sector in case of a floppy disk. The third parameter is the number of sectors that we wish to read. This parameter is specified as 1 since the boot sector occupies only a single sector. The last parameter is the address of a buffer/variable that would collect the data that is read from the floppy. Here we have passed the address of the boot structure variable **b**. As a result, the structure variable would be setup with the contents of the boot sector data at the end of the function call.

Once the contents of the boot sector have been read into the structure variable **b** we have displayed the first few of them on the screen using **printf( )**. If you wish you can print the rest of the contents as well.

The *ReadSector( )* Function

With the preliminaries over let us now concentrate on the real stuff in this program, i.e. the **ReadSector( )** function. This function begins by making a call to the **CreateFile( )** API function as shown below:

```
h = CreateFile  ( src, GENERIC_READ,
                  FILE_SHARE_READ, 0, OPEN_EXISTING, 0, 0 ) ;
```

The **CreateFile( )** API function is very versatile. Anytime we are to communicate with a device we have to firstly call this API function. The **CreateFile( )** function opens the specified device as a file. Windows treats all devices just like files on disk. Reading from this file means reading from the device.

The **CreateFile( )** API function takes several parameters. The first parameter is the string specifying the device to be opened. The second parameter is a set of flags that are used to specify the desired access to the file (representing the device) about to be opened. By specifying the **GENERIC_READ** flag we have indicated that we just wish to read from the file (device). The third parameter specifies the sharing access for the file (device). Since floppy drive is a shared resource across all the running applications we have specified the **FILE_SHARE_READ** flag. In general while interacting with any hardware the sharing flag for the file (device) must always be set to this value since every piece of hardware is shared amongst all the running applications. The fourth parameter indicates security access for the file (device). Since we are not concerned with security here we have specified the value as **0**. The fifth parameter specifies what action to take if

the file already exists. When using **CreateFile( )** for device access we must always specify this parameter as **OPEN_EXISTING**. Since the floppy disk file was already opened by the OS a long time back during the booting. The remaining two parameters are not used when using **CreateFile( )** API function for device access. Hence we have passed a **0** value for them. If the call to **CreateFile( )** succeeds then we obtain a handle to the file (device).

The device file mechanism allows us to read from the file (device) by setting the file pointer using the **SetFilePointer( )** API function and then reading the file using the **ReadFile( )** API function. Since every sector is **512** bytes long, to read from the n^{th} sector we need to set the file pointer to the **512 * n** bytes from the start of the file. The first parameter to **SetFilePointer( )** is the handle of the device file that we obtained by calling the **CreateFile( )** function. The second parameter is the byte offset from where the reading is to begin. This second parameter is relative to the third parameter. We have specified the third parameter as **FILE_BEGIN** which means the byte offset is relative to the start of the file.

To actually read from the device file we have made a call to the **ReadFile( )** API function. The **ReadFile( )** function is very easy to use. The first parameter is the handle of the file (device), the second parameter is the address of a buffer where the read contents should be dumped. The third parameter is the count of bytes that have to be read. We have specified the value as **512 * num** so as to read **num** sectors. The fourth parameter to **ReadFile( )** is the address of an **unsigned int** variable which is set up with the count of bytes that the function was successfully able to read. Lastly, once our work with the device is over we should close the file (device) using the **CloseHandle( )** API function.

Though **ReadSector( )** doesn't need it, there does exist a counterpart of the **ReadFile( )** function. Its name is **WriteFile( )**. This API function can be used to write to the file (device). The parameters of **WriteFile( )** are same as that of **ReadFile( )**. Note

that when **WriteFile( )** is to be used we need to specify the **GENERIC_WRITE** flag in the call to **CreateFile( )** API function. Given below is the code of **WriteSector( )** function that works exactly opposite to the **ReadSector( )** function.

```
void WriteSector ( char *src, int ss, int num, void* buff )
{
    HANDLE h ;
    unsigned int br ;
    h = CreateFile  ( src, GENERIC_WRITE,
                      FILE_SHARE_WRITE, 0, OPEN_EXISTING, 0, 0 ) ;
    SetFilePointer ( h, ( ss * 512 ), NULL, FILE_BEGIN ) ;
    WriteFile ( h, buff, 512 * num, &br, NULL )  )
    CloseHandle ( h ) ;
}
```

Accessing Other Storage Devices

Note that the mechanism of reading from or writing to any device remains standard under Windows. We simply need to change the string that specifies the device. Here are some sample calls for reading/writing from/to various devices:

```
ReadSector ( "\\\\.\\a:", 0, 1, &b ) ;  /* reading from 2nd floppy drive */
ReadSector ( "\\\\.\\d:", 0, 1, buffer ) ;  /* reading from a CD-ROM drive */
WriteSector ( "\\\\.\\c:", 0, 1, &b ) ;  /* writing to a hard disk */
ReadSector ( "\\\\.\\physicaldrive0", 0, 1, &b ) ;  /* reading partition table */
```

Here are a few interesting points that you must note.

(a) If we are to read from the second floppy drive we should replace **A:** with **B:** while calling **ReadSector( )**.

(b) To read from storage devices like hard disk drive or CD-ROM or ZIP drive, etc. use the string with appropriate drive letter. The string can be in the range \\.**C:** to \\.**Z:**.

(c) To read from the CD-ROM just specify the drive letter of the drive. Note that CD-ROMs follow a different storage organization known as CD File System (CDFS).

(d) The hard disk is often divided into multiple partitions. Details like the place at which each partition begins and ends, the size of each partition, whether it is a bootable partition or not, etc. are stored in a table on the disk. This table is often called 'Partition Table'. If we are to read the partition table contents we can do so by using the string **\\.\physicaldrive0**.

(e) Using **\\.\physicaldrive0** we can also read contents of any other parts of the disk. Here **0** represents the first hard disk in the system. If we are to read from the second hard disk we need to use **1** in place of **0**.

Communication with Keyboard

Like mouse messages there also exist messages for keyboard. These are **WM_KEYDOWN**, **WM_KEYUP** and **WM_CHAR**. Of these, **WM_KEYDOWN** and **WM_KEYUP** are sent to an application (which has the input focus) whenever the key is pressed and released respectively. The additional information in case of these messages is the code of the key being pressed or released. When we tackle **WM_KEYDOWN** or **WM_KEYUP** we need to ourselves check the status of toggle keys like NumLock and CapsLock and shift keys like Ctrl, Alt and Shift. If we wish to avoid all this checking we can tackle the **WM_CHAR** message instead.

What is mentioned above is the normal procedure followed by most Windows applications. However, if we wish to go a step further and deal with the keyboard we need to tackle it differently. For example, suppose we are to perform one of the following jobs:

(a) Once you hit any key CapsLock should become on. Once it becomes on it should remain permanently on.

(b) If we hit a key once it should appear twice on the screen.
(c) If we hit a key A then B should appear on the screen, if we hit a B then C should occur and so on.

Note that all these effects should work on a system-wide basis for all Win32 applications. To be able to achieve these effect we need understand two important mechanisms—'Dynamic Linking' and 'Windows Hooks'. Let us understand these mechanisms one by one.

Dynamic Linking

As we saw in Chapter 16, Windows permits linking of libraries stored in a .DLL file during execution. A .DLL file is a binary file that cannot execute on its own. It contains functions that can be shared between several applications running in memory.

Windows Hooks

As the name suggests, the hook mechanism permits us to intercept and alter the flow of messages in the OS before they reach the application. Since hooks are used to alter the messaging mechanism on a system-wide basis the code for hooking has to be written in a DLL. The hooking mechanism involves writing a hook procedure in a DLL file and registering this procedure with the OS. Since the DLL cannot execute on its own we need a separate program that would load and execute the DLL.

For different messages there are different types of hooks. For example, for keyboard messages there is a keyboard hook, for mouse messages there is mouse hook, etc. You can refer MSDN for nearly a dozen more types of hooks. Here we would restrict our discussion only to the keyboard hook.

Before we proceed to write our own hook procedure let us understand the normal working of the keyboard messages. This is illustrated in Figure 19.7.

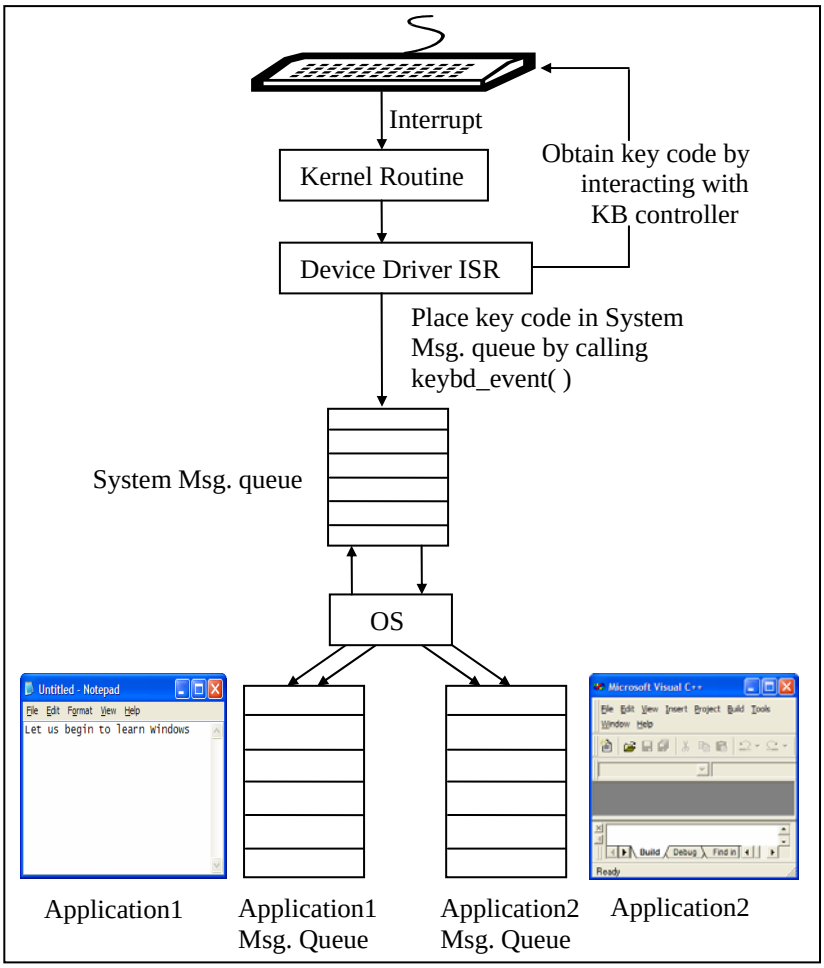

Figure 19.7

With reference to Figure 19.7 here is a list of steps that are carried out when we press a key from the keyboard:

(a) On pressing a key an interrupt occurs and the corresponding kernel routine gets called.
(b) The kernel routine calls the ISR of the keyboard device driver.
(c) The ISR communicates with the keyboard controller and obtains the code of the key pressed.
(d) The ISR calls a OS function **keybd_event( )** to post the key code to the System Message Queue.
(e) The OS retrieves the message from the System Message Queue and posts it into the message queue of the application with regard to which the key has been pressed.

Let us now see what needs to be done if we are to alter this procedure. We simply need to register our hook procedure with the OS. As a result, our hook procedure would receive the message before it is dispatched to the appropriate Application Message Queue. Since our hook procedure gets a first shot at the message it can now alter the working in the following three ways:

(a) It can suppress the message altogether
(b) It can change the message
(c) It can post more messages into the System Message Queue using the **keybd_event( )** function.

Let us now put all this theory into practice by writing a few programs.

Caps Locked, Permanently

Let us now write a program that keeps the CapsLock permanently on. This effect would come into being when the first key is hit subsequent to the execution of our program. In fact there would be two programs:

(a) A DLL containing a hook procedure that achieves the CapsLock effect.
(b) An application EXE which loads the DLL in memory.

Given below is the source code of the DLL program.

```
/* hook.c */

# include <windows.h>

static HHOOK hkb = NULL ;
HANDLE h ;

BOOL __stdcall DllMain ( HANDLE hModule, DWORD ul_reason_for_call,
                         LPVOID lpReserved )
{
    h = hModule ;
    return TRUE ;
}

BOOL __declspec ( dllexport ) installhook( )
{
    hkb = SetWindowsHookEx ( WH_KEYBOARD,
                    ( HOOKPROC ) KeyboardProc, ( HINSTANCE ) h, 0 ) ;
    if ( hkb == NULL )
        return FALSE ;

    return TRUE ;
}

LRESULT __declspec ( dllexport ) __stdcall KeyboardProc ( int nCode,
                                    WPARAM wParam, LPARAM lParam )
{
    short int state ;

    if ( nCode < 0 )
        return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;

    if ( ( nCode == HC_ACTION ) &&
                    ( ( DWORD ) lParam & 0x40000000 ) )
    {
        state = GetKeyState ( VK_CAPITAL ) ;
        if ( (state & 1 )== 0)  /* if off */
```

```
        {
            keybd_event ( VK_CAPITAL , 0,
                        KEYEVENTF_EXTENDEDKEY, 0 ) ;
            keybd_event ( VK_CAPITAL , 0,
            KEYEVENTF_EXTENDEDKEY | KEYEVENTF_KEYUP, 0 ) ;
        }
    }
    return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;
}

BOOL __declspec ( dllexport )  removehook( )
{
    return UnhookWindowsHookEx ( hkb ) ;
}
```

Follow the steps mentioned below to create this program:

(a) Select 'File | New' option to start a new project in VC++.
(b) From the 'Project' tab select 'Win32 Dynamic-Link Library' and click on the 'Next' button.
(c) In the 'Win32 Dynamic-link Library Step 1 of 1' select "An empty DLL project" and click on the 'Finish' button.
(d) Select 'File | New' option.
(e) From the 'File' tab select 'C++ source file' and give the file name as 'hook.c'. Type the code listed above in this file.
(f) Compile the program to generate the .DLL file.

Note that this program doesn't contain **WinMain( )** since the program on compilation should not execute on its own. It has been replaced by a function called **DllMain( )**. This function acts as entry point of the DLL program. It gets called when the DLL is loaded or unloaded.

When the application loads the DLL the **DllMain( )** function would be called. In this function we have merely stored the handle to the DLL that has been loaded in memory into a global variable **h** for later use.

Those functions in a DLL that can be called from outside it are called exported functions. Our DLL contains three such functions—**installhook( )**, **removehook( )** and **KeyboardProc( )**. To indicate to the compiler that a function in a DLL is an exported function we have to pre-qualify it with **__declspec (dllexport)**. These functions would be called from the second program. This second program is a normal GUI application created in the same way that we did applications in Chapters 17 and 18. The handlers for messages **WM_CREATE** and **WM_DESTROY** are given below:

```
/* capslocked.c */

HINSTANCE h ;

void OnCreate ( HWND hWnd )
{
    BOOL ( CALLBACK *p )( ) ;

    h = LoadLibrary ( "hook.dll" ) ;
    if ( h != NULL )
    {
        p = GetProcAddress ( h, "installhook" ) ;
        ( *p )( ) ;  /* calls installhoook( ) function */
    }
}

void OnDestroy ( HWND hWnd )
{
    BOOL ( CALLBACK *p )( ) ;

    p = GetProcAddress ( h, "removehook" ) ;
    ( *p )( ) ;  /* calls removehoook( ) function */

    FreeLibrary ( h ) ;
    PostQuitMessage ( 0 ) ;
}
```

As we know, the **OnCreate( )** and **OnDestroy( )** handlers would be called when the **WM_CREATE** and **WM_DESTROY** messages arrive respectively. In **OnCreate( )** we have loaded the DLL containing the hook procedure. To do this we have called the **LoadLibrary( )** API function. Once the DLL is loaded we have obtained the address of the exported function **installhook( )** using the **GetProcAddress( )** API function. The returned address is stored in **p**, where **p** is a pointer to the **installhook( )** function. Using this pointer we have then called the **installhook( )** function.

In the **installhook( )** function we have called the API function **SetWindowsHookEx( )** to register our hook procedure with the OS as shown below:

```
hkb = SetWindowsHookEx ( WH_KEYBOARD,
                         ( HOOKPROC ) KeyboardProc, ( HINSTANCE ) h, 0 ) ;
```

Here the first parameter is the type of hook that we wish to register, whereas the second parameter is the address of our hook procedure **KeyboardProc( )**. **hkb** stores the handle of the hook installed.

From now on whenever a keyboard message is retrieved by the OS from the System Message Queue the message is firstly passed to our hook procedure, i.e. to **KeyboardProc( )** function. Inside this function we have written code to ensure that the CapsLock always remains on. To begin with we have checked whether **nCode** parameter is less than **0**. If it so then it necessary to call the next hook procedure. The MSDN documentation suggests that "if code is less than zero, the hook procedure must pass the message to the **CallNextHookEx( )** function without further processing and should return the value returned by **CallNextHookEx( )**".

Note that there can be several hook procedures installed by different programs, thus forming a chain of hook procedures. These hook procedures always get called in an order that is

opposite to their order of installation. This means the last hook procedure installed is the first one to get called.

If the **nCode** parameter contains a value **HC_ACTION** it means that the message that was just removed form the system message queue was a keyboard message. If it is so, then we have checked the previous state of the key before the message was sent. If the state of the key was 'depressed' (30^{th} bit of **lParam** is 1) then we have obtained the state of the CapsLock key by calling the **GetKeyState( )** API function. If it is off (0^{th} bit of **state** variable is 0) then we have turned on the CapsLock by simulating a keypress. For this simulation we have called the function **keybd_event( )** twice—first call is for pressing the CapsLock and second is for releasing it. Note that **keybd_event( )** creates a keyboard message from the parameters that we pass to it and posts it into the system message queue. The parameter VK_CAPITAL represents the code for the CapsLock key.

A word of caution! When we use **keybd_event( )** to post keyboard message for a simulated CapsLock keypress, once again our hook procedure would be called when these messages are retrieved from the system message queue. But this time the CapsLock would be on so we would end up passing control to the next hook procedure through a call to **CallNextHookEx( )**.

When we close the application window as usual the **OnDestroy( )** would be called. In this handler we have obtained the address of the **removehook( )** exported function and called it. In the **removehook( )** function we have unregistered our hook procedure by calling the **UnhookWindowsHookEx( )** API function. Note that to this function we have passed the handle to our hook. As a result our hook procedure is now removed from the hook chain. Hereafter the CapsLock would behave normally. Having unhooked our hook procedure the control would return to **OnDestroy( )** handler where we have promptly unload the DLL from memory by calling the **FreeLibrary( )** API function.

One last point about this program—the 'hook.dll' file should be copied into the directory of the application's EXE before executing the EXE.

Did You Press It TTwwiiccee….

With the power of windows hooks below your belt you are into the league of power programmers of Windows. So how about tasting the power some bit more. How about writing a program that would make every key pressed in any Windows application appear twice. Here is the code for the hook procedure.

```
LRESULT __declspec ( dllexport ) __stdcall KeyboardProc ( int nCode,
                                        WPARAM wParam, LPARAM lParam )
{
    static BYTE key  ;
    static BOOL flag = FALSE ;

    if ( nCode < 0 )
        return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;

    if ( ( nCode == HC_ACTION ) &&
        ( ( DWORD ) lParam & 0x80000000 ) == 0 )
    {
        if ( flag == FALSE )
        {
            key = wParam ;
            keybd_event ( key , 0, KEYEVENTF_EXTENDEDKEY, 0 ) ;
            flag = TRUE ;
        }
        else
        {
            if ( key == ( BYTE ) wParam )
                flag = FALSE ;
        }
    }
    return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;
```

```
}
```

In this hook procedure once again we have checked if the **nCode** parameter contains a value **HC_ACTION**. If it does then we have checked the present state of the key in question. If the present state of the key is 'pressed' (31^{th} bit of **lParam** is 0) then we have posted the message for the same key into the system message queue by calling the **keybd_event( )**. However, this may lead to a serious problem. Can you imagine which? The message that we post, once retrieved, would again bring the control to our hook procedure. Once again the conditions would become true and we would post the same message again. This would go on and on. This can be prevented by using a using a simple **flag** variable as shown in the code.

Note that the rest of the functions in the DLL file are exactly same as in the previous program. So also is the application program.

Mangling Keys

How about one more program to bolster your confidence? Let us try one that would mangle every key that is pressed. That is, convert an A to a B, B to C, C to D, etc. This would be fairly straight-forward. We simply have to increment the key code before posting it into the system message queue. Also, further processing of key has to be prevented. This can be achieved by simply returning a non-zero value from the hook procedure (thus bypassing the call to **CallNextHookEx( )**). This is shown in the following hook procedure.

```
LRESULT __declspec ( dllexport ) __stdcall KeyboardProc ( int nCode,
                                        WPARAM wParam, LPARAM lParam )
{
    static BYTE key ;
    static BOOL flag = FALSE ;
```

```
    if ( nCode < 0 )
        return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;

    if ( ( nCode == HC_ACTION ) &&
       ( ( DWORD ) lParam & 0x80000000 ) == 0 )
    {
        if ( flag == FALSE )
        {
            key = wParam ;
            key ++ ;
            keybd_event ( key , 0, KEYEVENTF_EXTENDEDKEY, 0 ) ;
            flag = TRUE ;
            return 1 ;
        }
        else
        {
            if ( key == ( BYTE ) wParam )
                flag = FALSE ;
        }
    }
    return CallNextHookEx ( hkb, nCode, wParam, lParam ) ;
}
```

KeyLogger

There are several malicious programs that are floating on the net that steal away your passwords. These programs keep a log of every key that is pressed while entering passwords or credit card numbers. These programs make use of windows hooks to trap every key that is pressed. With the knowledge that you have gained from the past three programs this may not be a big deal.

However, such key logger programs deviate from the ones that we developed in three fundamental ways:

(a) They do not pop any window on the screen; otherwise the program's presence would get detected.

(b) These programs also hide themselves from the Task Manager so that the user cannot terminate them.

(c) The logged keys are secretly sent over the net to the malicious users who write such programs. Once the logged keys are known it would be possible to break into the system.

Where is This Leading

Even for a moment do not create an impression in you mind that Windows Hooks are only for notorious activities. There are many good things that they can be put to use for. These activities include:

(a) Multimedia keyboards have special key like Cut, Copy, Paste, etc. Such keyboards also come with special programs which when installed know how to tackle these special keys. On pressing these keys these programs use the hook mechanism to place the simulated keys in the system message queue.

(b) Many demo programs once executed automatically move the mouse pointer to a menu or a toolbar or any such item to demonstrate some feature of the software. To manage these actions a windows hook called Journal hook is used.

(c) For physically impaired persons a keyboard can be simulated on the screen and the mouse clicks on this keyboard can be communicated to Windows as actual key hits. This again can be achieved using mouse and keyboard hook.

There can be many more such examples. But the above three I believe would be ample to prove to you the constructive side of the powerful mechanism called Windows Hooks.